

운영체제의 기초 — 수업 정리 노트

서울대학교 홍성수 교수님 강의 정리 | Ch1~Ch12 + ACID 보충 | 2026-06-11

01. Introduction — 운영체제 입문

📖 학습 목표

운영체제(OS)란 무엇이며 왜 필요한가? 어떻게 진화해 왔는가? 가장 원시적인 메모리 보호 메커니즘인 base/bound MMU를 통해 OS와 hardware의 협력을 이해.

I. What is an Operating System?

OS는 다음 세 가지 역할을 동시에 수행하는 소프트웨어입니다.

1) Coordinator — 조율자

여러 사용자/프로세스/응용이 한정된 자원(CPU, 메모리, 디스크 등)을 두고 경쟁할 때, OS가 충돌을 방지하고 공정하게 분배합니다.

- **CPU scheduling**: 누가 다음에 실행될지 결정
- **Memory management**: 누가 어디를 쓸지 결정
- **Lock 관리**: 동시 접근 방지

2) Illusion Generator — 환상 제조기

물리적 하드웨어를 더 좋은 가상 머신처럼 보이게 합니다.

- **Virtual memory**: 실제 메모리보다 큰 주소 공간이 있는 것처럼
- **Multitasking**: 여러 프로그램이 동시 실행되는 것처럼
- **GUI**: 복잡한 시스템 콜을 친숙한 창과 아이콘으로

3) Standard Library — 표준 라이브러리

공통적인 기능을 미리 만들어 응용에 제공.

- File system API (open, read, write)
- Network stack
- Inter-process communication

II. Evolution of Operating Systems — OS의 진화

Phase I (1950s): Single-program Batch

- ## Phase II (1960s): Multi-programmed Batch

- ### Phase III (1970s~): Time-sharing

- ### III. Base/Bound MMU — 가장 원시적인 메모리 보호

```

    Logical Addr (offset) → ① < Bound? → Yes → ② + Base → Physical
                                | No
                                ▼
                                Trap to OS

```

Base와 Bound는 privileged register: kernel만 수정 가능.

이 한계가 Chapter 10의 **multiple segments**, 더 나아가 **paging**으로 이어집니다.

⑤ 핵심 통찰

1. **OS**는 세 가지 역할의 동시 수행 — coordinator + illusion + library
2. **OS** 진화는 자원 활용도와 보호 요구의 진화 — CPU 활용 ↑, 보호 필요성 ↑, 응답성 ↑
3. **Base/Bound**는 모든 **MMU**의 출발점 — 단순하지만 sharing이 불가능한 하계가 다음 챕터로 이어짐

⚠ 흔한 오해

- “OS는 GUI다”: GUI는 illusion generator의 한 측면일 뿐. OS의 본질은 자원 관리.
 - “Time-sharing이 multiprogramming”: 다릅니다. Multiprogramming은 메모리에 여러 program, multiprocessing은 CPU에 여러 process, time-sharing은 interactive 응답성을 위한 fine-grained switching.
 - “Base/Bound가 modern OS에 쓰임”: 아닙니다. Modern OS는 paging + segmentation. Base/Bound는 역사적 출발점.
-

02. Hardware — 하드웨어 기반

📖 학습 목표

OS가 동작하는 hardware 환경을 이해. von Neumann architecture, FDE cycle, system bus, interrupt 메커니즘, hardware protection. 이 모든 것이 OS 기능의 토대.

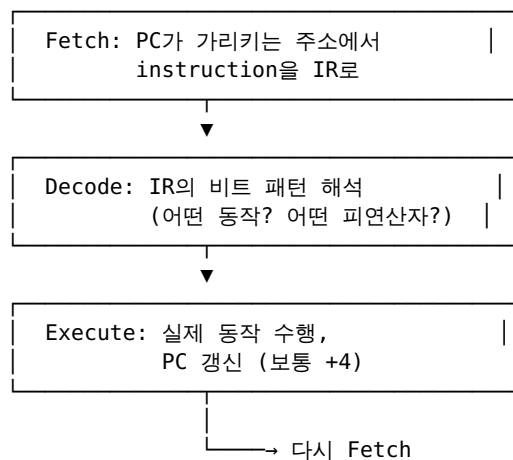
I. Von Neumann Architecture와 Stored-program Concept

Stored-program concept (von Neumann, 1945): - 프로그램(code)도 데이터처럼 메모리에 저장 - CPU가 메모리에서 명령어를 차례로 읽어 실행 - **Code**와 **data**가 같은 메모리 공간 공유 → 큰 유연성

이 아이디어가 모든 modern computer의 기반. Reprogrammable computer의 시초.

II. Fetch-Decode-Execute Cycle

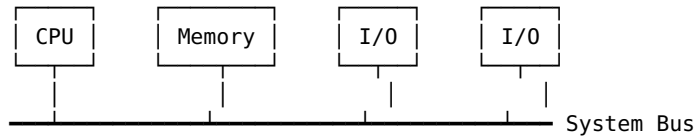
모든 instruction의 일생:



CPU 내부 registers: - **PC** (Program Counter): 다음 instruction 주소 - **IR** (Instruction Register): 현재 instruction - **General registers**: 데이터 보관

III. System Bus

CPU, memory, I/O가 통신하는 공용 통로.



Bus master: bus transaction을 시작할 수 있는 component - CPU는 항상 bus master - DMA controller도 bus master 가능 (CPU 없이 메모리 ↔ device 전송)

Bus의 세 가지 **line**: - **Address bus**: 어디로/어디서 - **Data bus**: 실제 데이터 - **Control bus**: read/write 등 명령

IV. Interrupt Mechanism

CPU가 다른 사건에 즉시 반응할 수 있게 하는 핵심 메커니즘.

Interrupt가 일어나면?

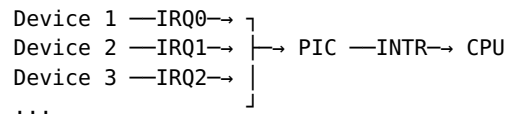
```
현재 instruction 실행 중...
↓
Interrupt 신호 감지 (다음 FDE 시작 전)
↓
① 현재 PC, register들을 stack에 저장
↓
② Interrupt vector table 조회
↓
③ Interrupt Handler로 PC 변경
↓
Handler 실행 (mode = kernel)
↓
④ iret: 저장된 PC, registers 복원
↓
원래 위치에서 실행 재개
```

Hardware vs Software Interrupt

구분	Hardware Interrupt	Software Interrupt (Trap)
동기성	Asynchronous	Synchronous
원인	외부 신호 (timer, disk 등)	명령어 실행 결과 (syscall, exception)
예시	Timer, keyboard, network	int 0x80, divide-by-zero

PIC (Programmable Interrupt Controller)

여러 I/O 장치의 IRQ를 CPU의 단일 INTR 핀에 다중화.



PIC가 priority를 매기고 어떤 interrupt를 먼저 보낼지 결정.

V. Hardware Protection — 4가지 보호

OS의 신뢰성은 hardware protection에 전적으로 의존. 4가지가 상호 의존적.

1. Dual Mode Operation (모든 보호의 기반)

Mode bit	의미
0	Kernel mode (모든 instr 가능)
1	User mode (제한된 instr만)

PSW (Processor Status Word)에 mode bit 저장.

Privileged instruction: kernel mode에서만 실행 가능 - I/O instruction - Halt - Modify privileged registers (base/bound, page table base, mode bit 자체 등)

User mode에서 privileged 시도 → **trap** → kernel이 처리.

2. I/O Protection

- 모든 I/O instruction은 privileged
- User process는 I/O를 직접 못 함
- **System call**을 통해서만 (mode switch가 일어남)

3. Memory Protection

- Base/Bound register 자체가 **privileged**
- 매 메모리 접근마다 MMU가 검사
- 다른 process나 OS 메모리 접근 시 trap

4. CPU Protection

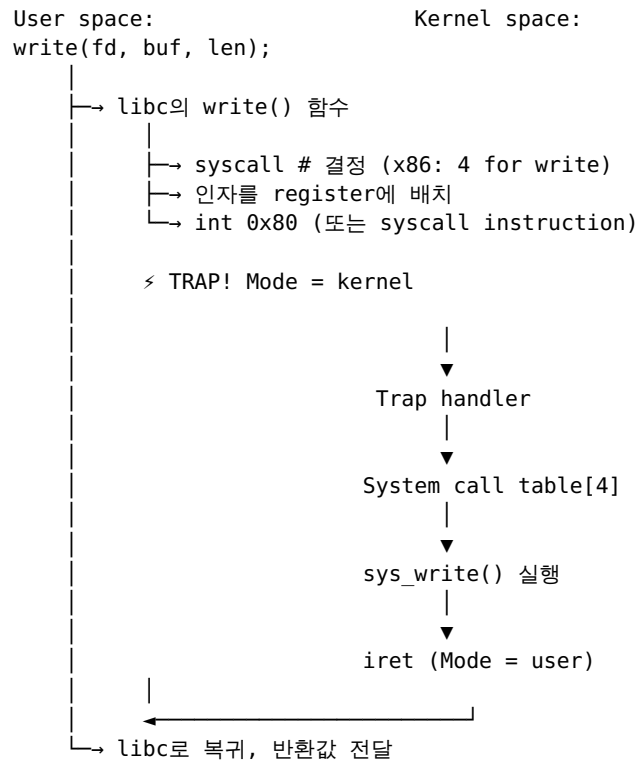
- **Hardware timer**가 일정 시간마다 interrupt 발생
- Kernel이 timer interrupt에서 scheduling 결정
- 무한 루프 process도 강제 회수 가능

왜 상호 의존적인가?

만약 dual mode가 없으면 user가 base/bound를 수정 가능 → memory protection 무력화
 Dual mode가 없으면 user가 timer를 끌 수 있음 → CPU protection 무력화 → **Dual mode가 모든 보호의 기반**

VI. System Call

User mode에서 kernel 기능을 호출하는 메커니즘.



System call은 **mode switch**를 동반하는 함수 호출.

🔍 핵심 통찰

1. **Stored-program concept**이 모든 것의 기반 — code와 data가 메모리에 함께
2. **FDE cycle**은 **instruction**의 일생 — interrupt가 이를 가로채는 메커니즘
3. **Hardware protection 4**가지는 상호 의존 — dual mode가 없으면 나머지 무의미
4. **PIC**은 여러 **device interrupt**를 다중화 — priority 기반
5. **System call = mode switch**를 동반하는 함수 호출 — user mode와 kernel mode를 잇는 다리

⚠️ 흔한 오해

- “**Interrupt**와 **trap**은 같다”: 비슷하지만 다릅니다. Interrupt는 비동기 (외부), trap은 동기 (명령어). 처리 메커니즘은 같지만 발생 원인이 다름.
- “**Dual mode**는 **software** 기능”: 아닙니다. **Hardware** 기능입니다. PSW의 mode bit는 CPU의 register.
- “**System call**이 일반 함수 호출보다 무조건 빠르다”: 아닙니다. **Mode switch overhead**가 큼 (수십~수백 cycles).

03. Runtime Stack — 런타임 스택

📖 학습 목표

함수 호출이 stack에서 어떻게 일어나는지 깊이 이해. x86 cdecl calling convention, stack frame, prologue/epilogue 패턴. 이게 모든 program 실행의 토대.

I. Why Stack?

함수 호출의 **LIFO** 특성: - 마지막에 호출된 함수가 먼저 반환됨 - 각 호출은 자기 지역 변수, 인자, return address가 필요 - Stack이 이 LIFO 패턴에 완벽

II. x86 cdecl Calling Convention

x86의 표준 calling convention. C compiler가 따름.

핵심 규칙

- **인자:** 모두 스택에 (오른쪽에서 왼쪽으로 push)
- **반환값:** `%eax`
- **Caller-saved:** `%eax, %ecx, %edx` — caller가 보존 책임
- **Callee-saved:** `%ebx, %esi, %edi, %ebp` — callee가 보존 책임

Stack Frame 구조

함수가 호출될 때마다 생기는 영역:

높은 주소 ↑ (스택은 위에서 아래로 자람)

arg N	ebp + 4 · (N+1)
...	
arg 2 (y)	ebp + 12
arg 1 (x)	ebp + 8
return address	ebp + 4
saved old ebp	← %ebp
local 1	ebp - 4
local 2	ebp - 8
...	
	← %esp (top)

낮은 주소 ↓

기억할 위치들: - 인자: `ebp+8, ebp+12, ebp+16, ...` - 지역변수: `ebp-4, ebp-8, ...` - 반환값: `%eax`

III. Prologue & Epilogue

Prologue (함수 시작)

```
pushl %ebp      ; 호출자의 ebp를 stack에 저장
movl %esp, %ebp ; ebp = 새 frame의 시작점
subl $N, %esp   ; 지역 변수 N bytes 공간 확보
```

Epilogue (함수 종료)

```

movl %ebp, %esp ; esp = frame 시작 (지역변수 영역 해제)
popl %ebp       ; 호출자의 ebp 복원
ret             ; return address pop → PC = return addr

```

또는 더 간결하게:

```

leave          ; movl %ebp,%esp + popl %ebp
ret

```

IV. 예제: add 함수 호출 추적

```

int add(int x, int y) {
    int z = x + y;
    return z;
}

int main() {
    int result = add(3, 4);
    return result;
}

```

main이 add를 호출하기 직전:

stack top		
4		← y 인자
3		← x 인자
ret		← call로 push된 return address

add의 prologue 후:

4		ebp+12
3		ebp+8
ret		ebp+4
old ebp		← ebp
z		ebp-4
		← esp

int z = x + y:

```

movl 8(%ebp), %eax ; x를 eax로
addl 12(%ebp), %eax ; eax += y
movl %eax, -4(%ebp) ; z = eax

```

return z:

```

movl -4(%ebp), %eax ; eax = z (반환값)
movl %ebp, %esp
popl %ebp
ret

```

V. leal 명령어 트릭

leal (Load Effective Address)은 메모리 접근 없이 주소 계산만:

```

leal 4(%eax, %ebx, 2), %ecx
; ecx = eax + ebx*2 + 4
; 메모리는 건드리지 않음

```


이걸로 빠른 산술 연산 가능: - `leal (%eax, %eax, 2), %eax` → $eax = 3 * eax$ -
`leal (%eax, %eax, 4), %eax` → $eax = 5 * eax$

🔍 핵심 통찰

1. **Stack frame**은 함수 호출의 활성 영역 — 호출마다 생성, 반환 시 소멸
2. 인자는 **ebp+8**부터, 지역변수는 **ebp-4**부터 — 외워야 할 핵심
3. **Prologue/Epilogue**가 **calling convention**의 실체 — 모든 함수가 이 패턴
4. **Stack**은 위에서 아래로 자람 — 새 frame은 더 낮은 주소에
5. **leal**은 산술 트릭 — 메모리 접근 없는 주소 계산기

⚠ 흔한 오해

- “**Heap**도 **stack**처럼 동작”: 아닙니다. **Stack**은 **LIFO**, heap은 임의 순서 (malloc/free).
- “**x86-64**도 **cdecl**”: 아닙니다. x86-64는 **System V AMD64 ABI** (인자 6개는 register, 그 이후만 stack).
- “**Return address**는 **callee**가 **push**”: 아닙니다. **call instruction**이 **push**, callee는 **ret**로 **pop**.

보충: Transaction & ACID

📖 학습 목표

Ch1의 ATM race condition을 일반화한 **transaction** 개념과 ACID 속성. 이게 Chapter 06 synchronization의 미리보기이자, 이후 모든 동시성 논의의 핵심 어휘.

I. Transaction이란?

Transaction: 하나의 논리적 작업 단위. - 여러 step으로 이루어져도 전체가 하나처럼 동작 - 예: 계좌 A에서 B로 이체 - Step 1: A에서 100 차감 - Step 2: B에 100 추가 - 둘 중 하나만 실행되면 안 됨 (돈이 사라지거나 생김)

II. ACID 속성

Atomicity (원자성)

전부 성공 **or** 전부 실패. 부분 실행 안 됨.

- 위 이체 예시: Step 1만 되고 Step 2 안 되면 → Step 1도 취소 (rollback)
- “All or nothing”

Consistency (일관성)

Transaction 전후로 **invariant** 유지. - 예: “모든 계좌 잔액 합은 일정”이 invariant라면, transaction은 이를 유지 - 응용 수준의 의미적 일관성

Isolation (격리성)

동시 실행되는 **transaction**끼리 간섭 **X**. - = race condition 방지 - 마치 transaction이 혼자 실행되는 것처럼 보임 - Ch1의 ATM 문제가 정확히 이걸 위반한 사례

Durability (영속성)

Commit된 후에는 결과가 영구적. - 시스템 충돌, 정전이 일어나도 결과 유지 - 일반적으로 disk write로 보장

III. ATM 예시 다시 보기

초기 잔액: 1000

T1 (서울 ATM):
read balance: 1000
subtract 300
write balance: 700

T2 (부산 ATM):
read balance: 1000
subtract 500
write balance: 500

Isolation 위반: T1과 T2가 race condition → 두 트랜잭션 결과 중 하나가 사라짐.

올바른 동작: 한 txn이 끝날 때까지 다른 txn은 시작 안 됨 (또는 read를 못 봄)

→ 이게 **mutex/lock**의 동기. Ch6에서 자세히.

IV. Serializability

Serializable execution: 동시 실행 결과가 어떤 순차 실행과 같음. - 예: T1 → T2 순서와 같은 결과거나, T2 → T1 순서와 같은 결과 - 이게 isolation의 형식적 정의

왜 동시 실행해도 **OK**? Performance — 한 번에 하나씩 처리하면 너무 느림. 동시 실행하되 결과는 순차처럼.

☯ 핵심 통찰

1. **Transaction** = 하나의 논리적 단위 (여러 step도 하나처럼)
 2. **ACID 4**가지 속성 — 모두 보장되어야 신뢰성 있는 transaction
 3. **Isolation** = **race condition** 방지 — Ch1 ATM 문제의 정확한 진단
 4. Ch6의 **mutex/semaphore**가 **isolation**의 **mechanism**
 5. **Durability**는 **file system, transaction log**의 영역
-

04. Process and Threads — 프로세스와 스레드

📖 학습 목표

OS의 가장 중요한 추상화인 **process**를 깊이 이해. PCB, state transition, context switching, fork/exec/wait, 그리고 더 가벼운 추상화인 **thread**.

I. Process란?

Process = 실행 중인 program. **Program** ≠ **Process**: - Program: 디스크의 파일 (정적) - Process: 실행 중인 program 인스턴스 (동적, runtime 상태)

같은 program이 여러 process가 될 수 있음 (예: 여러 terminal 창에서 같은 shell).

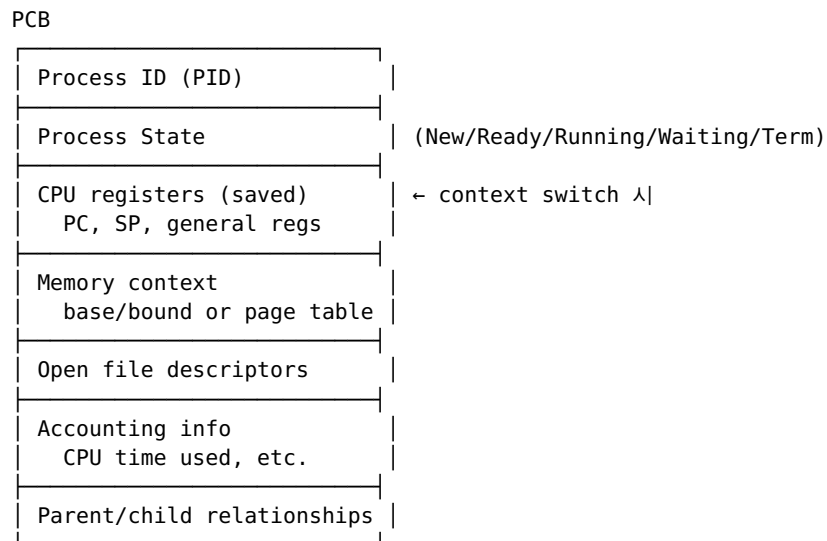
Process의 두 측면

1. 자원 소유의 단위: 메모리, open files, 시간 quantum 등
2. 실행의 단위: PC, registers, stack

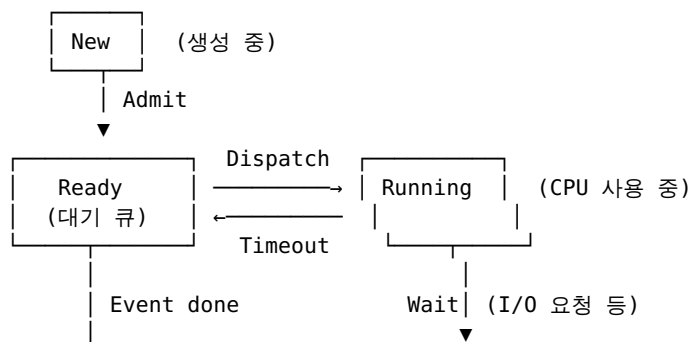
이 두 측면이 **thread**에서 분리됩니다.

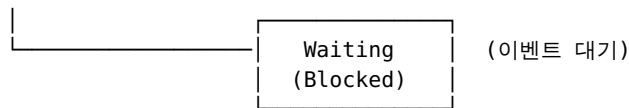
II. PCB (Process Control Block)

각 process마다 kernel이 유지하는 자료구조. process의 모든 정보.



III. Process States and Transitions





Running — Exit —> Terminated

각 전이의 트리거가 시험에 잘 나옴: - New → Ready: admit (메모리 할당 등 완료) - Ready → Running: dispatch (스케줄러가 선택) - Running → Ready: timeout (timer interrupt 또는 더 높은 priority가 ready) - Running → Waiting: I/O 요청, semaphore 대기 등 - Waiting → Ready: event 발생 (I/O 완료, semaphore 신호) - Running → Terminated: exit(), abort()

IV. Context Switching

Process 전환의 메커니즘.

1. 현재 process의 register들을 PCB에 save
2. Scheduler가 다음 process 선택 (policy)
3. 선택된 process의 PCB에서 register restore
4. PC를 새 process의 instruction으로

Dispatcher (mechanism) vs Scheduler (policy): - Dispatcher: 실제 전환 작업 (위 1-4단계) - Scheduler: 누구를 선택할지 결정 (Ch5)

Context switch는 순수 **overhead** (실제 일은 안 하고 전환만). 그래서 너무 자주 일어나면 안 됨.

V. fork() — Process 생성

Unix의 process 생성 메커니즘. 현재 **process**를 복제:

```

pid_t pid = fork();
if (pid == 0) {
    // 자식 process
    printf("I am child, PID=%d\n", getpid());
} else if (pid > 0) {
    // 부모 process
    printf("I am parent, child PID=%d\n", pid);
} else {
    // fork 실패
}
  
```

중요: fork()는 한 번 호출되어 두 번 반환! - 부모에게: child의 PID (양수) - 자식에게: 0 - 실패: -1 (한 process로만 돌아옴)

COW (Copy-on-Write)

fork()의 본래 의미는 완전한 복사지만, 실제 구현은: 1. **Page table**만 복사, page 자체는 부모와 공유 2. 모든 page를 **read-only**로 표시 3. 쓰기 시도 시 **trap** → 그 page만 진짜 복사 후 쓰기 허용

→ 대부분의 fork는 exec()가 뒤따라 곧바로 모든 page를 버리니까, 미리 복사 안 하는 게 효율적.

VI. exec() — Program 교체

```

execv("/bin/ls", argv);
// 이 줄 이후는 실행 안 됨 (성공 시)
// 현재 process가 ls 프로그램으로 바뀜

```

- 현재 **process**의 **memory image**를 새 프로그램으로 교체
- PID는 유지, 같은 PCB
- 보통 fork() 직후 자식이 exec() 호출

VII. wait() — 자식 종료 대기

```

pid_t cpid = fork();
if (cpid == 0) {
    // 자식
    execv("/bin/ls", argv);
} else {
    // 부모
    int status;
    wait(&status); // 자식이 끝날 때까지 대기
}

```

wait()를 안 하면 **zombie process** 발생: - 자식이 종료해도 **PCB**는 남아있음 (부모가 종료 코드 읽도록) - 부모가 wait() 안 하면 PCB 영원히 안 사라짐 → 자원 누수

Orphan process: 부모가 먼저 죽은 자식. init (PID 1)이 입양해서 wait() 해 줌.

VIII. Threads — 더 가벼운 실행 단위

Process는 자원 소유 + 실행 단위. **Thread**는 실행 단위만.

Process vs Thread

항목	Process	Thread
주소 공간	독립	공유 (같은 process 내)
스택	자기 것	자기 것 (스택만!)
Code/Data/Heap	독립	공유
생성 비용	비쌈 (PCB, page table)	쌈 (TCB만)
통신	IPC 필요	공유 메모리 직접
보호	OS가 격리	없음 (같은 process 내)

Thread의 장점

- 생성/전환 비용 ↓
- 공유 메모리로 통신 쉬움
- **Multi-core** 활용 (한 process가 여러 코어 사용)

Thread의 단점

- 공유 메모리로 **race condition** 위험 (Ch6에서 다룸)
- 한 **thread**의 **segfault**가 전체 **process kill**
- 격리 부족

IX. Thread 구현 3가지

ULT (User-Level Threads)

- **Library**가 **user space**에서 관리 (예: pthreads의 일부 구현)
- **Kernel**은 **process**만 봄 (thread 모름)

장점: - Mode switch 없음 (빠른 thread switch) - App-specific scheduling

단점: - 한 **thread**의 **blocking syscall**이 전체 **process** 멈춤 (kernel은 thread 모르니까) - Multi-core 활용 불가 (kernel이 process를 한 코어에만)

KLT (Kernel-Level Threads)

- **Kernel**이 **thread**를 직접 관리
- Linux의 일반적 thread

장점: - Thread blocking이 다른 thread 영향 X - Multi-core 활용 가능

단점: - **Thread switch**마다 2번 **mode switch** - Thread 생성도 syscall

Combined (M:N)

- **M user threads** → **N kernel threads**로 매핑
- 양쪽의 장점 결합
- 구현 복잡도 ↑
- 예: Windows NT의 fiber, Solaris의 LWP

X. Pthreads API

POSIX threads — Unix의 표준 thread API.

```
#include <pthread.h>

pthread_t tid;

// 생성
pthread_create(&tid, NULL, thread_func, arg);

// 종료 대기
pthread_join(tid, &retval);

// 자기 종료
pthread_exit(retval);

// 자기 thread id
pthread_self();
```

TLS (Thread-Local Storage)

각 thread가 자기만의 전역 변수를 가질 수 있음:

```
__thread int my_var; // GCC 확장
```

각 thread에서 my_var은 서로 독립적인 값.

🔗 핵심 통찰

1. **Process** = 자원 + 실행, **Thread** = 실행만
2. **PCB**는 **process**의 모든 정보 — context switch가 PCB save/restore
3. **State transition**은 6가지 — 각 트리거를 외워야
4. **fork**는 한 번 호출, 두 번 반환 — Unix만의 특이한 모델
5. **COW**로 **fork** 최적화 — 대부분의 fork가 exec로 이어지니까
6. **Thread** 3가지 구현은 **trade-off** — mode switch vs blocking vs multi-core

⚠️ 흔한 오해

- “**Process**와 **program**은 같다”: 아닙니다. Program은 정적, process는 동적.
- “**fork**는 두 번 호출된다”: 아닙니다. 한 번 호출되어 두 번 반환.
- “**Thread**는 자원도 공유”: 자원의 일부만 공유 (memory, files). **Stack**은 **thread**별로 독립.
- “**ULT**가 항상 더 빠르다”: Thread switch는 빠르지만, **blocking syscall**이 치명적.

05. CPU Scheduling — CPU 스케줄링

📖 학습 목표

여러 process가 CPU를 두고 경쟁할 때 누구를 다음에 실행시킬지 결정하는 **policy**를 다룸. FIFO부터 Linux CFS까지의 진화.

I. Scheduling의 기초

CPU Burst

Process의 실행은 **CPU burst + I/O burst**의 반복:

└─CPU─┐I/O└─CPU─┐I/O└─CPU─┐I/O└─┐

짧음 길음 짧음 길음 ...

스케줄러가 정렬하는 것은 **process**가 아니라 **CPU burst**입니다.

Scheduling Metrics

지표	정의	좋음
Throughput	단위 시간당 완료된 process 수	높을수록
Turnaround time	도착부터 완료까지	낮을수록
Waiting time	Ready queue에서 대기한 시간	낮을수록
Response time	도착부터 첫 응답까지	낮을수록

II. Scheduling Algorithms

1) FIFO (FCFS — First Come First Served)

도착 순서로 끝까지 실행.

문제: **Convoy Effect** - 긴 process 뒤에 짧은 process가 줄을 서서 평균 **waiting time**이 매우 큼

예시: P1(burst 24), P2(3), P3(3) 순으로 도착 - Avg waiting = $(0 + 24 + 27) / 3 = 17$ - 반대 순서 P2, P3, P1이면 $= (0 + 3 + 6) / 3 = 3!$

2) SJF (Shortest Job First) / SRTF (Shortest Remaining Time First)

짧은 **burst**부터 실행.

- **SJF**: 비선점 (시작하면 끝까지)
- **SRTF**: 선점 (더 짧은 게 오면 교체)

이론적으로 평균 **waiting time** 최적 (Optimality).

문제: 미래의 **burst** 길이를 모름.

Exponential Moving Average

과거 burst들로 다음 burst 예측:

$$\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$$

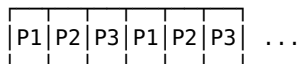
- t_n : 실제 n 번째 burst 길이
- τ_n : 예측치
- α : smoothing factor (보통 0.5~0.8)

예시 ($\alpha = 0.5$, $\tau_0 = 10$, burst = 6, 4, 6, 4, 13...): - $\tau_1 = 0.5 \cdot 6 + 0.5 \cdot 10 = 8$ - $\tau_2 = 0.5 \cdot 4 + 0.5 \cdot 8 = 6$ - $\tau_3 = 0.5 \cdot 6 + 0.5 \cdot 6 = 6$ - $\tau_4 = 0.5 \cdot 4 + 0.5 \cdot 6 = 5$ - $\tau_5 = 0.5 \cdot 13 + 0.5 \cdot 5 = 9$

→ 예측이 부드럽게 변함 (지수 가중 평균).

3) Round Robin (RR)

각 process에 일정 **time slice** 할당, 순환:



Time slice 크기의 **trade-off**: - 너무 길면 → FIFO에 가까움 (응답성 ↓) - 너무 짧으면 → context switch overhead가 너무 큼

적정값: 보통 10~100ms.

4) MLFQ (Multi-Level Feedback Queue)

Priority 별 여러 큐. 적응적:

- Q0 (highest, slice=1) → [P1, P2]
- Q1 (slice=2) → [P3]
- Q2 (slice=4) → [P4]
- Q3 (lowest, slice=8) → [P5]

규칙: 1. 새 process는 최고 **priority**, time slice = $2^0 = 1$ 2. Time slice 다 쓰고 안 끝나면 → priority 강등, time slice 2배 3. **I/O**로 **block**되면 **priority** 유지 (= I/O 친화적 process 우대)

이게 자연스럽게: - **I/O-intensive** → 항상 짧게 실행 → 높은 priority 유지 → 응답성 ↑ - **CPU-intensive** → 길게 실행 → 점점 강등 → 영향 최소화

5) Fair Share Scheduling

각 process에 **weight** 비례 CPU 시간.

GPS (Generalized Processor Sharing) — 이상

Weight 1:2:3인 process A, B, C가 있다면
매 순간 1/6, 2/6, 3/6 비율로 CPU

무한히 작은 **quantum**으로 weight 비례. 구현 불가능 (이상).

WRR (Weighted Round Robin)

GPS의 근사 1: **Time slice**를 **weight** 비례로.

- A=10ms, B=20ms, C=30ms씩 받음
- 단순, O(1)
- 큰 time slice면 fairness가 떨어짐

WFQ (Weighted Fair Queuing)

GPS의 근사 2: **Virtual runtime** 기반.

- 각 task에 VR 부여, VR = 실제 시간 / weight
- **VR**이 가장 작은 (가장 뒤쳐진) **task**부터 실행
- Network에서 패킷 스케줄링에 많이 사용

6) Linux CFS (Completely Fair Scheduler)

Linux 2.6.23부터의 표준 스케줄러. WFQ의 변형.

Virtual Runtime

$$VR(\tau_i, t) = (W_0 / W(\tau_i)) \cdot PR(\tau_i, t)$$

- W_0 : nice 0의 weight (1024)
- $W(\tau_i)$: task의 weight (nice에 따라)
- PR: 실제 실행 시간

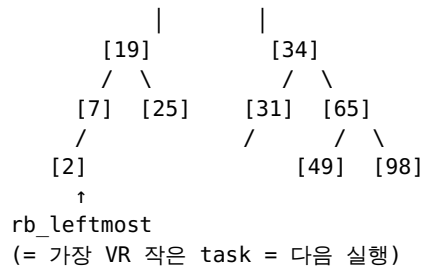
Weight가 클수록 **VR**이 천천히 증가 → 더 자주 실행.

Time Slice

$$TS_i = (W_i / S_\Phi) \cdot P$$

- P: target latency (예: 6ms)
- S_Φ : 모든 active task의 weight 합

Run Queue: Red-Black Tree



- $O(\log n)$ 삽입/삭제
- **Leftmost pointer가 $O(1)$ lookup**

Nice 10% Rule

- Nice 1단계 차이 = **CPU 사용량 $\pm 10\%$**
- Weight table:
 - nice -20: 88761
 - nice 0: 1024
 - nice +19: 15

III. Scheduling 진화 요약

SRTF (optimal) → 미래 모름
 ↓
 RR → time slice 크기 문제
 ↓
 MLFQ → fair share 안 됨
 ↓
 GPS (ideal) → 구현 불가
 ↓
 WRR (round robin 근사) | WFQ (virtual time 근사)
 ↓
 CFS (RB-tree, virtual runtime)

🌀 핵심 통찰

1. **Scheduling의 대상은 burst, not process**
2. 모든 알고리즘은 **GPS**의 근사 — 진정한 fairness는 불가능
3. **MLFQ**의 천재성: 명시적 분류 없이 행동만 보고 I/O-intensive 우대
4. **CFS = WFQ + RB-tree** — Linux 표준
5. **Virtual Runtime**이 **fair share**의 핵심 개념 — 받아야 할 시간 대비 실제 받은 시간

⚠️ 흔한 오해

- “**SJF**가 항상 최적”: 이론적으로 평균 waiting time 최적이지만, 미래 모름이 치명적.
- “**Time slice**는 짧을수록 좋다”: 너무 짧으면 **context switch overhead**가 압도.
- “**CFS**는 **SJF**”: 아닙니다. **Fair share** — weight 비례 분배.
- “**Higher nice = higher priority**”: 반대입니다. Nice가 클수록 priority 낮음 (“nicer” → 양보).

06. Synchronization — 동기화

📖 학습 목표

여러 process/thread가 공유 자원에 접근할 때 발생하는 **race condition**과 해결책. Semaphore, condition variable, monitor의 추상화 사다리.

I. Race Condition

공유 자원에 동시 접근 시 타이밍에 따라 결과가 달라지는 현상.

고전 예시: 카운터 증가

```
counter++; // 한 줄 같지만 실제로는 3 instruction:
           // load counter to reg
           // increment reg
           // store reg to counter
```

두 thread가 동시 실행하면:

```
T1: load counter (0)
T1: increment (reg=1)
T1: store counter (1)

T2: load counter (0) ← 잘못된 값!
T2: increment (reg=1)
T2: store counter (1)
```

결과: counter = 1, 그러나 기대는 2

비유: ATM 동시 인출 (Ch1 ACID 복습)

- T1: 서울 ATM에서 300원 인출
- T2: 부산 ATM에서 500원 인출
- 둘 다 동시 시작 → 한 transaction의 결과가 사라짐

II. Critical Section과 Mutual Exclusion

Critical section: 동시에 한 thread만 실행 가능해야 하는 코드 영역.

Mutual exclusion: critical section에 한 번에 한 thread만 진입하도록 보장.

단순한 시도와 실패

```
// 시도 1: 단순 flag (실패)
int locked = 0;
while (locked) ; // wait
locked = 1;      // ← 두 thread가 동시에 여기 도달 가능!
... critical ...
locked = 0;
```

문제: while 후 locked=1이 atomic 아님. 두 thread가 모두 통과 가능.

Disable Interrupt (single CPU only)

```

disable_interrupt();
... critical ...
enable_interrupt();

```

단일 CPU에서만 동작. Multicore에서는 다른 CPU가 동시 접근 가능.

III. Semaphore

Dijkstra가 1965년 제안. 가장 우아한 동기화 메커니즘.

정의

Semaphore S: 음이 아닌 정수 값 + 두 atomic 연산

P(S): wait until $S > 0$, then $S--$
V(S): $S++$

- **P:** “wait” 또는 “down” (네덜란드어 Proberen)
- **V:** “signal” 또는 “up” (네덜란드어 Verhogen)

Binary vs Counting Semaphore

	Binary	Counting
값	0 또는 1	임의 정수
용도	Mutual exclusion (= mutex)	자원 N개 관리

기본 사용 패턴

```

Semaphore mutex = 1;

P(mutex);
... critical section ...
V(mutex);

```

Producer/Consumer Pattern ★

가장 중요한 패턴. 두 가지 semaphore:

```

Semaphore buf_avail = N;    // 빈 슬롯
Semaphore data_avail = 0;   // 데이터 있는 슬롯
Buffer buf[N];

Producer() {
    while (1) {
        item = produce();
        P(buf_avail);       // 빈 슬롯 대기
        buf[in] = item;
        V(data_avail);      // 데이터 슬롯 증가
    }
}

Consumer() {
    while (1) {
        P(data_avail);      // 데이터 대기
        item = buf[out];
        V(buf_avail);       // 빈 슬롯 증가
    }
}

```

```

        consume(item);
    }
}

```

핵심 패턴: - 자원 생성 시 V() (signal) - 자원 소비 시 P() (wait) -
 Producer/Consumer가 대각선으로 P/V 짝맞춤

Semaphore의 두 역할

1. **Mutual exclusion**: 초기값 1 → critical section 보호
2. **Scheduling**: 초기값 0 → 한 thread가 다른 thread를 기다림

IV. Semaphore 구현

Single-CPU 구현

```

struct Semaphore {
    int count;
    Queue waiting;
};

void P(Semaphore *s) {
    disable_interrupt(); // critical
    s->count--;
    if (s->count < 0) {
        enqueue(s->waiting, current_thread);
        block();
    }
    enable_interrupt();
}

void V(Semaphore *s) {
    disable_interrupt();
    s->count++;
    if (s->count <= 0) {
        Thread *t = dequeue(s->waiting);
        wakeup(t);
    }
    enable_interrupt();
}

```

Multi-CPU 구현: Test-And-Set

```

// Atomic instruction: TAS(addr)
//   old = *addr; *addr = 1; return old;
// 모두 atomic하게 (hardware support)

void P(Semaphore *s) {
    while (TAS(&s->lock)) ; // spin
    s->count--;
    ... (위와 비슷)
    s->lock = 0;
}

```

Multi-CPU에서 **disable_interrupt**만으로는 불충분 — 다른 CPU에서 동시 접근 가능. **Hardware atomic instruction** 필수.

V. Pthread API

Mutex (= Binary Semaphore)

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_lock(&m);
... critical ...
pthread_mutex_unlock(&m);
```

POSIX Semaphore

```
sem_t s;
sem_init(&s, 0, 1); // 마지막 인자 = 초기값
sem_wait(&s);      // P
sem_post(&s);      // V
sem_destroy(&s);
```

VI. Condition Variable

Semaphore보다 더 일반적. 임의의 조건을 기다림.

동작

- **wait(cv, mutex):** ① mutex unlock ② sleep ③ signal 시 깨어남 ④ mutex 재획득
- **signal(cv):** 하나 깨움
- **broadcast(cv):** 모두 깨움

표준 사용 패턴

```
pthread_mutex_lock(&mutex);
while (!condition)
    pthread_cond_wait(&cv, &mutex);
// 여기서 condition이 보장됨
... critical ...
pthread_mutex_unlock(&mutex);
```

중요한 점들:

1. **wait가 mutex를 인자로 받음** — unlock과 wait이 atomic해야 함. 안 그러면 **missed wakeup**:

```
wait의 잘못된 구현:
unlock(mutex);
// ← 여기서 다른 thread가 signal하면 놓침!
sleep();
```

2. 반드시 **while: Mesa semantics**에서는 signal 후 조건이 다시 깨질 수 있음:
 - Signal 받음 → mutex 기다림 → 다른 thread가 먼저 들어와 조건 변경 → 다시 wait해야

Producer/Consumer with CV

```
pthread_mutex_t m;
```

```

pthread_cond_t not_full, not_empty;
int count = 0;

Producer() {
    pthread_mutex_lock(&m);
    while (count == N) pthread_cond_wait(&not_full, &m);
    buf[in] = produce();
    count++;
    pthread_cond_signal(&not_empty);
    pthread_mutex_unlock(&m);
}

Consumer() {
    pthread_mutex_lock(&m);
    while (count == 0) pthread_cond_wait(&not_empty, &m);
    item = buf[out];
    count--;
    pthread_cond_signal(&not_full);
    pthread_mutex_unlock(&m);
}

```

VII. Monitor

가장 높은 추상화. Lock이 자동.

정의

Monitor: 데이터 + 그 데이터를 다루는 procedures + condition variables
의 캡슐. 한 번에 한 **thread**만 **monitor** 내부에 있음 (자동 mutex).

```

Monitor BoundedBuffer {
    int buf[N], count = 0;
    Condition not_full, not_empty;

    procedure put(item) {
        while (count == N) wait(not_full);
        buf[in] = item; count++;
        signal(not_empty);
    }

    procedure get() {
        while (count == 0) wait(not_empty);
        item = buf[out]; count--;
        signal(not_full);
        return item;
    }
}

```

명시적 lock 없음 — 컴파일러/언어가 자동 처리.

Mesa vs Hoare Semantics

	Mesa	Hoare
Signal 후 사용처 코드	signaler가 계속, waiter는 큐로 대부분 현대 시스템 while (조건 재확인)	waiter가 즉시 실행 이론적 if 가능

Java Monitor

```
synchronized void method() {  
    while (!condition) wait();  
    ... critical ...  
    notifyAll();  
}
```

synchronized가 자동 lock, wait/notify가 condition variable.

☯ 핵심 통찰

1. **Race condition** = 타이밍에 의존하는 버그
2. **Atomicity**와 **mutual exclusion**은 다른 차원 — atomic은 instruction, mutex는 critical section
3. **Semaphore**의 두 역할: mutex (init=1) + scheduling (init=0)
4. **Producer/Consumer**는 두 **semaphore** — 자원 생성/소비의 대칭
5. **Condition variable**의 **wait**가 **mutex**를 받는 이유: missed wakeup 방지
6. 항상 **while (not if)** — Mesa semantics
7. **Monitor**는 자동 **lock** + **condition variable**의 추상화

⚠ 흔한 오해

- “**Race condition**은 **multicore**에서만”: 아닙니다. Single core에서도 preemption으로 발생.
- “**Disable interrupt**면 모든 **race** 해결”: 단일 CPU만. Multicore에서는 hardware atomic 필요.
- “**if**로 **wait** 검사도 가능”: Mesa semantics에서는 반드시 **while**.
- “**Signal**하면 즉시 **waiter** 실행”: Mesa에서는 ready queue로만. 실제 실행은 나중에.

07. Deadlocks — 교착 상태

📖 학습 목표

여러 thread가 lock을 잡으려다 서로 기다리며 영원히 정지하는 deadlock의 4가지 조건과 prevention/detection 전략.

I. Deadlock이란?

모든 **process**가 다른 **process**가 가진 자원을 기다려 진행 불가능한 상태.

고전 예시: 두 **mutex**

Thread 1:	Thread 2:
lock(A);	lock(B);
lock(B); ← 대기...	lock(A); ← 대기...

T1은 B를 기다리고, T2는 A를 기다림. 영원히 진행 불가.

II. 4 Necessary Conditions (Coffman, 1971)

다음 4가지가 모두 성립해야 deadlock 발생:

1. **Mutual Exclusion** — 자원을 한 번에 하나만 가질 수 있음 (공유 불가)
2. **No Preemption** — 자원을 강제로 회수 불가
3. **Hold and Wait** — 자원을 가진 채 다른 자원 기다림
4. **Circular Wait** — wait 그래프에 순환 존재

중요: 4가지가 필요조건이지 충분조건이 아님. 모두 성립해도 deadlock 안 일어날 수 있지만, 하나라도 없으면 **deadlock** 절대 안 일어남.

→ **Prevention** 전략: 4가지 중 하나를 깨라.

III. Prevention 전략

1. Mutex 깨기 — 자원 공유 가능하게

대부분 자원이 본질적으로 mutex 필요 → 비실용적.

2. No Preemption 깨기 — 강제 회수 허용

일부 자원만 가능 (CPU, memory). Disk, printer는 어려움.

3. Hold and Wait 깨기 — 한꺼번에 다 요청

자원 활용도 떨어짐, 미래 필요량 예측 어려움.

4. Circular Wait 깨기 — Ordered Request ★

가장 실용적인 전략. 자원에 전역 순서 부여, 항상 그 순서로 요청.

Resource A < Resource B < Resource C

Always: lock A first, then B, then C

Never: lock B first, then A

→ Circular wait 절대 불가.

IV. Banker's Algorithm

자원 할당이 **safe state**를 유지하는지 검사 후 허용.

핵심 개념

- **Safe state**: 어떤 순서로든 모든 process의 max를 만족시킬 수 있음
- **Unsafe state**: deadlock 가능성 있는 상태 (반드시 deadlock은 아님)

자료구조

- Available[m]: 가용 자원
- Max[n][m]: 최대 필요량

- Allocation[n][m]: 현재 할당
- Need[n][m] = Max - Allocation

Safety Algorithm

Work = Available

Finish[i] = false for all i

REPEAT:

Find i: Finish[i]==false AND Need[i] ≤ Work

if found:

Work += Allocation[i]

Finish[i] = true

else:

break

if all Finish[i] == true: SAFE

else: UNSAFE

예시

	Max	Allocation	Need = Max-Alloc	Available
P0	7 5 3	0 1 0	7 4 3	3 3 2
P1	3 2 2	2 0 0	1 2 2	
P2	9 0 2	3 0 2	6 0 0	
P3	2 2 2	2 1 1	0 1 1	
P4	4 3 3	0 0 2	4 3 1	

Safe sequence: (P1, P3, P4, P0, P2)

V. Detection과 Recovery

Detection: Banker's와 유사, **Need** 대신 **Request** 사용.

Recovery 방법: 1. **Process termination** (다 죽이기 또는 하나씩) 2. **Resource preemption** (rollback)

Recovery는 위험합니다 — kill된 process가 weird state면 data 손상.

VI. Practice의 현실

“Deadlock is one area where there is a strong theory but it is almost completely ignored in practice.”

대부분의 OS는 deadlock을 무시 (ostrich algorithm). Mission-critical은 prevention by design (ordered request).

☪ 핵심 통찰

1. 4 조건은 필요조건 — 하나라도 깨면 절대 X
2. **Ordered request**가 가장 실용적 **prevention**
3. **Banker's**는 이론적 우아함, 실용성 ↓
4. **Practice**는 무시가 흔함

⚠ 흔한 오해

- “**Cycle**이 있으면 항상 **deadlock**”: Multi-instance 자원에서는 cycle이 있어도 deadlock 아닐 수 있음.
- “**Safe state**면 무조건 안전”: Safe state는 deadlock 회피 가능 보장. 그러나 unsafe도 항상 deadlock은 아님.

08. GNU Linker — 링커

📖 학습 목표

여러 .c 파일이 어떻게 하나의 실행 파일이 되는가? Compile pipeline, ELF format, symbol resolution, relocation, static vs dynamic linking.

I. Compile Pipeline

gcc -o prog p1.c p2.c 한 줄 뒤의 4단계:

```
p1.c -cpp→ p1.i -cc1→ p1.s -as→ p1.o ↗
                                     | ↘ ld → prog
p2.c -cpp→ p2.i -cc1→ p2.s -as→ p2.o ↘
```

- **cpp**: C preprocessor (매크로, #include)
- **cc1**: C compiler (.c → .s)
- **as**: assembler (.s → .o)
- **ld**: linker (.o → 실행 파일)

II. Object File과 ELF

세 종류 object file:

종류	확장자	설명
Relocatable	.o	다른 .o와 결합 가능
Executable	(없음)	메모리에 직접 로드 가능
Shared	.so	Dynamic linking용

모두 **ELF** (Executable and Linkable Format) 포맷.

ELF Object File 구조

ELF header		
Program header table		
.text section		← code
.rodata section		← read-only data
.data section		← initialized globals

.bss section	← uninit globals (header only!)
.symtab section	← symbol table
.rel.text section	
.rel.data section	
Section header table	

BSS의 트릭

int x; (초기화 안 된 전역)은 .bss section의 **header**에만 기록. 실제 파일에는 공간 안 차지! 로드 시 OS가 0으로 초기화.

→ “Better Save Space”.

III. Linker의 두 작업

작업 1: Symbol Resolution

각 심볼 참조를 정확히 하나의 정의에 연결.

Strong vs Weak symbols: - **Strong:** function + initialized global - **Weak:** uninitialized global

Linker rules: 1. **Multiple strong** → error 2. **Strong + weak** → **strong** 선택 3. **Multiple weak** → 임의 선택

예시 — 위험한 케이스:

```
// file1.c
double x = 1.0; // strong, 8 bytes

// file2.c
int x;           // weak, 4 bytes
int y = 100;

void main() {
    x = 2;       // 8 bytes 쓰는데... y 망가짐!
}
```

Linker는 **type checking** 안 함 → 위험.

작업 2: Relocation

여러 .o의 같은 종류 section을 단일 **section**으로 병합 + 상대 주소를 절대 주소로.

IV. Static vs Dynamic Linking

Static Linking (.a)

- Library의 .o를 archive로 묶음
- Link 시 사용된 .o만 executable에 복사

- 결과: 큰 executable

```
ar rcs libfoo.a foo1.o foo2.o
gcc -static -o prog main.o -L. -lfoo
```

Command line 순서 주의:

```
gcc -static -o prog -L. -lfoo main.o # ✗ undefined ref!
gcc -static -o prog main.o -L. -lfoo # ✓ library를 뒤에
```

Dynamic Linking (.so)

- Library를 런타임에 **link**
- 모든 process가 한 copy 공유 → 메모리 절약

```
gcc -shared -fpic -o libfoo.so foo1.c foo2.c
gcc -o prog main.c -L. -lfoo
LD_LIBRARY_PATH=. ./prog
```

-fpic: Position-Independent Code. Shared library에 필수.

Runtime Dynamic Linking

```
#include <dlfcn.h>

void *handle = dlopen("libfoo.so", RTLD_LAZY);
int (*func)(int) = dlsym(handle, "my_func");
int result = func(42);
dlclose(handle);
```

용도: plugins, lazy loading, hot reload.

V. Loading

OS가 ELF 실행 파일을 메모리에 올리는 과정:

1. exec() 시스템 콜
2. ELF header 읽기
3. Program header에 따라 segment 메모리에 매핑
4. Heap, stack 준비
5. Dynamic linker 실행 (ld-linux.so)
6. Dynamic library resolve + relocate
7. PC를 entry point로 → process 시작

🔗 핵심 통찰

1. **Compile pipeline**: cpp → cc1 → as → ld
2. **ELF**는 모든 **object file**의 표준
3. **Linker**의 두 작업: symbol resolution + relocation
4. **BSS**는 공간을 안 차지 — header만
5. **Static vs Dynamic linking trade-off**: 크기/업데이트 vs 메모리 절약/유연성
6. **Library**는 **command line** 뒤에

⚠️ 흔한 오해

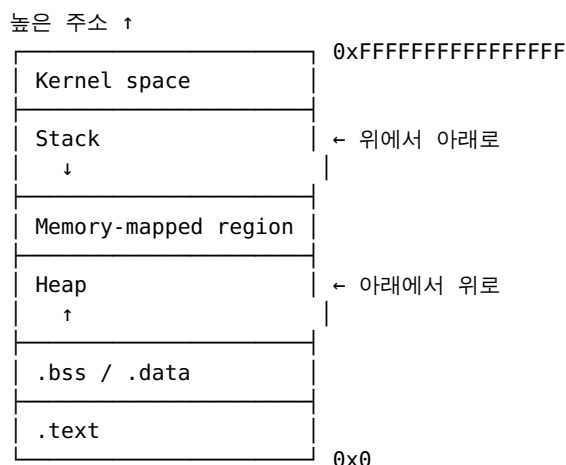
- “gcc가 컴파일러”: **Compile driver** — 여러 도구를 호출.
- “**Linker가 type checking**”: 안 합니다.
- “**Static이 항상 더 빠름**”: 시작 시간만. 메모리는 동적이 효율적.

09. Dynamic Memory Allocation — 동적 메모리 할당

📖 학습 목표

malloc/free가 내부적으로 어떻게 동작하는가? Heap 구조, fragmentation, free list 구현 방식, GC.

I. x86-64 Linux Memory Layout



II. Heap의 기본

malloc(size), free(ptr)로 동적 관리.

sbrk()로 Heap 확장

```
void *sbrk(intptr_t incr);
// brk pointer를 incr만큼 이동
```

malloc()은 내부적으로 sbrk 호출하지만, 한 번에 큰 chunk 받아두고 잘게 나눠 줌.

III. Heap의 5가지 설계 질문

1. **Free할 때 크기를 어떻게 아는가?** → **Header**
2. **Free block을 어떻게 추적?** → 3가지 방법
3. 여러 후보 중 어떤 **block** 선택? → fit 정책
4. 남는 공간 처리? → **Split**
5. **Free** 시 어떻게 **insert**? → LIFO 또는 address-ordered

Q1: Header

header (size, info)	payload (사용자 영역)
------------------------	---------------------

Q2: Free Block 추적

Implicit List

모든 block을 size로 순회. 단순하지만 $O(\text{전체})$.

Explicit List

Free block에 prev/next pointer. $O(\text{가용})$.

Segregated List

Size class별 list. 거의 $O(1)$. 현대 표준.

Q3: 어떤 block?

정책	의미
First-fit	충분히 큰 첫 block
Best-fit	가장 잘 맞는 (최소 fragmentation)
Next-fit	First-fit, 이전 위치부터
Worst-fit	가장 큰 block

Q5: Coalescing 4 Cases

Case	좌 (prev)	우 (next)	결과
1	Alloc	Alloc	자기만 free
2	Alloc	Free	자기 + 우
3	Free	Alloc	좌 + 자기
4	Free	Free	좌 + 자기 + 우

Boundary Tag: block 끝에 size 복사. 이게 있어야 bidirectional coalescing.

header (size)	payload	footer (size)
------------------	---------	------------------

IV. Fragmentation

Internal

할당된 block 안의 미사용. Alignment, minimum block size 등.

External

전체 free는 충분하나 연속된 큰 영역이 없음.

```
[used][free 100][used][free 200][used][free 50]
총 free: 350, 400 요청 → ✕
```

V. Linux Memory Allocation

크기	메커니즘
< mmap threshold	Heap (sbrk)
≥ mmap threshold	AMO (anonymous mmap)

Heap 기반

빠름, 작은 할당에 효율. Free 후 mapping 재사용. 단점: free 후 OS 반환 안 됨.

AMO 기반

큰 크기에 사용. Free 시 즉시 OS 반환. 단점: 매번 system call.

VI. Garbage Collection

Mark and Sweep

1. **Mark**: reachable 객체 모두 표시
2. **Sweep**: 표시 안 된 것 free

장점: 정확, 순환 처리. 단점: stop-the-world.

Reference Counter

각 객체에 ref count.

```
obj->refcount++;
if (--obj->refcount == 0) free(obj);
```

장점: 즉시. 단점: 순환 참조 못 풀음.

🔗 핵심 통찰

1. **Heap**의 5가지 설계 질문이 모든 allocator 결정
2. **Boundary tag**로 **bidirectional coalescing**
3. **Segregated list**가 현대 표준
4. **Internal vs External fragmentation**은 다른 차원
5. **Linux**는 **heap + AMO** 두 메커니즘
6. **GC**: mark-sweep vs reference counter

⚠️ 흔한 오해

- “**malloc**이 항상 **system call**”: 큰 chunk 받아두고 분배.
- “**free**하면 **OS**에 반환”: allocator의 free list로만 (mmap 제외).
- “**Best-fit**이 항상 최선”: 검색 비용 큼.
- “**GC**가 **manual**보다 무조건 좋다”: Real-time에 부적합.

10. Segmentation and Paging — 세그멘테이션과 페이징

📖 학습 목표

Chapter 1의 base/bound가 어떻게 진화해 현대 MMU의 기반이 되었는지. Segmentation, paging, paged segmentation, TLB.

I. Segmentation

Base/Bound의 한계 (1 segment per process)를 해결.

Segment Table:

Seg #	Base	Bound	Prot	
0	4000	6FF	10	← code
1	0000	4FF	11	← data
2	3000	FFF	00	
3	8000	FFF	11	← stack

Logical address: [seg#][offset]

장단점

장점: 논리적 단위 분리, sharing 가능 단점: External fragmentation

II. Paging

모든 **chunk**를 같은 크기 (**page**)로 → external fragmentation 해결.

Logical: [page#][offset] (예: 20-bit page#, 12-bit offset → 4KB page)

Physical: [frame#][offset]

Page Table[page#] = frame#

Page offset은 변환 안 됨.

장단점

장점: External fragmentation 해결, 할당 단순, swapping 단순 단점: 1. Access overhead (매번 page table → 2배) 2. Page table size (32-bit, 1KB page → 16MB)

III. Paged Segmentation

각 segment마다 별도 page table.

Logical: [seg#][page#][offset]

Segment table → Page table → Frame.

장점

- Segment의 논리적 분리 + Page의 fragmentation 해결
- 불필요한 segment는 page table 안 만들
- 사용자에게 paging 노출 안 됨

예시: **System/370**

24-bit: 4-bit seg + 8-bit page + 12-bit offset - 16 segments, 256 pages/segment, 4KB pages

IV. TLB (Translation Lookaside Buffer)

TLB (small, fast cache):

VPN	PFN
0x1234	0x5678
...	...

대개 32~512 entries

EAT (Effective Access Time)

$$t_{em} = \alpha \cdot (t_c + t_m) + (1-\alpha) \cdot (2t_c + t_m) = t_c + t_m + (1-\alpha) \cdot t_m$$

- t_c : TLB 검색 (예: 10ns)
- t_m : 메모리 접근 (예: 100ns)
- α : TLB hit ratio

예시 ($\alpha = 0.98$): $t_{em} = 10 + 100 + 0.02 \cdot 100 = \mathbf{112 \text{ ns}}$

TLB 없으면 200ns → 약 **1.78배** 가속.

Context Switch 처리

새 process는 다른 page table → TLB 무효:

1. **Flush all**: 단순하지만 비쌈
2. **ASID**: 각 entry에 process ID. 자동 미스

대부분 modern CPU는 ASID 지원.

Hardware vs Software TLB

- **Hardware-managed**: CPU가 직접 page table 검색 (x86, ARM)
- **Software-managed**: TLB miss → OS handler (MIPS, SPARC)

대부분 hardware가 표준.

🔗 핵심 통찰

1. 각 기법은 이전의 한계 해결: base/bound → segmentation → paging → P+S
2. **Page offset**은 변환 안 됨 — 핵심 디자인
3. **Paging**의 두 문제: access overhead (→ TLB), table size (→ multi-level)
4. **P+S = segment**의 논리성 + **page**의 균등성
5. **TLB hit ratio 98%**로 거의 메모리 속도
6. **ASID**로 context switch TLB flush 회피

⚠️ 흔한 오해

- “**Page**는 항상 **4KB**”: 보통은. 하지만 2MB, 1GB huge page도.
 - “**Paging**이 **segmentation**보다 우월”: 다른 장점. Modern OS는 둘 다.
 - “**TLB hit**이 **100%** 가능”: 첫 접근은 miss. 95~99%가 한계.
-

11. Demand Paging — 요구 페이징

📖 학습 목표

프로세스의 일부만 메모리에 올려두고 필요할 때 가져오는 demand paging.
Locality, page fault, replacement, working set, thrashing.

I. Virtual Memory의 환상

📌 **Knuth의 90/10 법칙**: 시간의 90%가 코드의 10%에서.

이 **locality** 덕분에: - 프로세스의 일부만 메모리에 있어도 대부분 hit - 실제 메모리 크기보다 큰 가상 주소 공간 가능 - 여러 **process** 메모리 공존 (multiprogramming)

Locality 두 종류

- **Temporal**: 한 번 접근한 주소를 또 (loop 변수)
- **Spatial**: 인접 주소도 접근 (array 순회)

II. Page Fault Handling

Page Fault란?

PTE의 **valid bit = 0**인 page에 접근 → trap.

처리 흐름

1. Process가 page에 접근

2. MMU가 PTE 확인 - valid bit = 0
3. Trap → kernel page fault handler
4. Handler 확인:
 - 진짜 fault인가? (valid 주소?)
 - SIGSEGV면 process kill
5. 빈 frame 할당
 - 없으면 victim 선택 + page replacement
 - Dirty page면 disk write-back
6. 디스크에서 page 읽기 (← 가장 느림, ms)
7. PTE 업데이트: PFN 채우기, valid bit = 1
8. TLB에 entry 추가
9. 중단된 instruction 재시작

TLB Fault vs Page Fault

- **TLB fault:** PTE는 있지만 TLB에 없음. 빠름.
- **Page fault:** page가 메모리에 없음. 느림 (disk I/O).
- 모든 page fault는 TLB fault 동반
- TLB fault라고 다 page fault 아님

III. Page Replacement Algorithms

OPT (Optimal)

미래에 가장 늦게 쓸 **page** evict. 이론적 최적. 구현 불가.

LRU (Least Recently Used)

가장 오래 안 쓴 page evict.

문제: 정확한 구현이 비쌈.

FIFO

가장 먼저 들어온 page evict. 단순하지만 locality 무시.

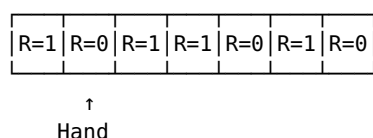
Belady's anomaly: frame을 늘려도 page fault 증가 (FIFO에만).

Stack Algorithm

LRU 같이 **frame** 늘리면 **page set**이 **superset**인 알고리즘. Belady's anomaly 없음.

Clock Algorithm (Second Chance) ★

LRU의 효율적 근사. **Reference bit** 활용:



Hand가 가리키는 page:
R=0? → evict
R=1? → R=0으로 set, hand 다음으로

장점: O(1) 평균 단점: 모든 R=1이면 FIFO처럼 동작

Enhanced Clock

(Reference, Dirty) 4가지 조합: - (0, 0): 가장 좋은 victim - (0, 1): write-back 필요 - (1, 0): used but clean - (1, 1): 가장 나쁜 victim

IV. Working Set Model

Peter Denning 제안. Thrashing 방지.

정의

Working set $W(t, \tau)$: 시간 t 의 직전 τ 초 동안 참조한 page들.

Working Set Strategy

Process는 **working set**이 메모리에 있어야 실행 가능. 안 들어가면 swap out.

Balance Set

Working set이 fit하는 **active processes** 집합.

→ Thrashing 방지 + multiprogramming degree 최대.

V. Thrashing

Page fault가 너무 많아 진전이 없는 상태.

Memory pressure ↑
↓
Replacement 빈번
↓
Page들이 곧 다시 필요해 page fault
↓
디스크 I/O로 시간 다 보냄
↓
CPU 활용도 폭락

OOM Killer

Linux의 최후 수단. 메모리 진짜 없을 때 low-priority process kill.

VI. mmap

File을 메모리 공간에 매핑.

```
void *p = mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
```

Minor vs Major Page Fault

- **Minor**: anonymous page 첫 접근. Disk I/O 없음. μs .
- **Major**: file-backed 또는 swap out된 page. **Disk I/O** 동반. ms (1000 배).

VII. Multi-level Page Table

64-bit 가상 주소 공간에서 단일 page table 비현실적.

x86-64의 4-level Paging

Virtual address (48-bit):

9b	9b	9b	9b	12b
PML4	PDPT	PD	PT	offset

PML4 → PDPT → PD → PT → Frame

- 4단계 lookup → **4번의 메모리 접근** (TLB miss 시)
- **TLB가 결정적으로 중요**

Hashed/Inverted Page Table

대안: - **Hashed**: VPN을 hash → bucket → PTE chain - **Inverted**: frame 당 하나의 entry

☯ 핵심 통찰

1. **Demand paging**의 환상은 **locality** 덕분
2. **Page fault** 흐름: trap → handler → frame → disk → PTE update → restart
3. **TLB fault** ≠ **Page fault**
4. **OPT**는 비교 기준, **Clock**이 실용
5. **LRU**만이 **stack algorithm**
6. **Working set** 모델로 **thrashing** 방지
7. **Minor vs Major page fault**: disk I/O 동반 여부
8. **x86-64**는 **4-level paging**

⚠ 흔한 오해

- “**Page fault**는 무조건 나쁨”: 처음 접근은 어차피 fault. Minor fault는 빠름.
- “**LRU**가 가장 좋음”: OPT가 최적, LRU는 근사.
- “**Belady’s anomaly**는 모든 알고리즘”: **FIFO**만.

12. I/O Devices and Device Drivers

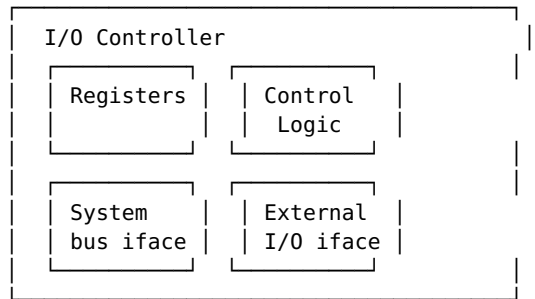
— I/O 장치와 디바이스 드라이버

📅 학습 목표

I/O 장치의 구조와 OS가 다양한 장치를 통합 추상화로 다루는 방법.

I. I/O Hardware

I/O Controller 구조



DMA (Direct Memory Access)

CPU 없이 device ↔ memory 직접 전송.

1. CPU가 DMA controller에 명령
2. CPU는 다른 일
3. DMA controller가 직접 전송
4. 완료 후 CPU에 interrupt

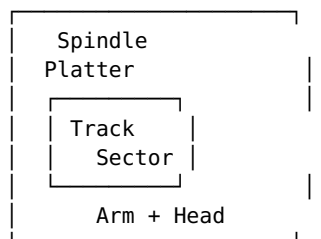
II. Three Types of I/O Devices

Character Devices

- Byte 단위
- mouse, keyboard, terminal

Block Devices

Disk Drive



4단계: 1. Head 선택 2. Track으로 이동 (**Seek**: 0~50ms) 3. Sector 회전 대기 (**Rotational delay**: 0~16ms) 4. Read/write (**Transfer**: 8~40μs)

Seek + rotation이 **transfer**의 **1000배** 느림 → disk scheduling 중요.

Flash (SSD)

Page (2/4/8 KB) ← read/write 단위
Block (64/128 pages) ← erase 단위

동작	시간
Read	20μs
Write	200μs
Erase	2000μs

두 제약: 1. **Erase-before-write** 2. **Wear-out**

→ **Log-structured** file system, wear leveling.

Network Devices

- Packet 단위
- Socket API

III. Device Drivers

OS가 다양한 device를 통합 추상화로 다루기 위한 소프트웨어 계층.

Device File

Unix “everything is a file”:

/dev/sda ← block device
/dev/sda1 ← partition
/dev/tty0 ← character device
/dev/null ← null device

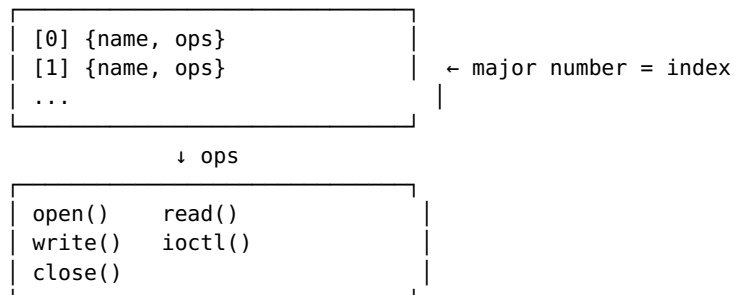
Major/Minor Number

```
$ ls -l /dev/sda
brw-rw---- 1 root disk 8, 0 ...
```

└──┬── Minor (instance)
└──┬── Major (driver)
└──┬── b/c (block/character)

Character Device Driver 구조

cdevsw table:



Block Device Driver 구조

Character + **buffer cache**:

```
bdevsw table → ops → request strategy
                      ↓
                  I/O queue
                      ↓
                  device
```

Buffer cache: 같은 block 반복 접근 시 hit

IV. Two-Level Interrupt Handling

ISR이 길면 응답성 ↓ → 두 단계 분리.

Top Half (ISR)

- 짧음, interrupt disabled
- Device ack, bottom half 활성화

Bottom Half

- 길어도 됨, interrupt enabled
- Deferred execution
- 실제 데이터 처리

```
Interrupt 발생!
↓
ISR (top half) – 매우 짧음
  1. Ack to device
  2. Bottom half 활성화
  3. 종료 (interrupt enable)
↓
... 정상 process 또는 다른 interrupt ...
↓
Bottom half 실행 – 길어도 OK
```

V. I/O Routine 4가지 조합

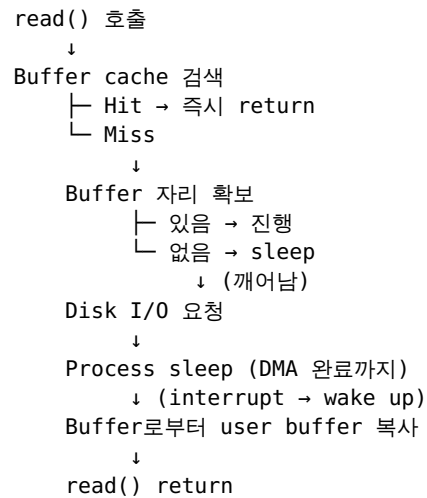
Polling vs Interrupt-Driven

	Polling	Interrupt-Driven
방법	반복 검사	sleep + wake
CPU	낭비	효율
응답성	즉시	latency

Blocking vs Non-Blocking

	Blocking	Non-Blocking
방법	대기	즉시 return

Block Device의 Interrupt + Blocking Flow



두 번 **sleep** 가능: 1. Buffer 자리 대기 2. Disk I/O 완료 대기

☯ 핵심 통찰

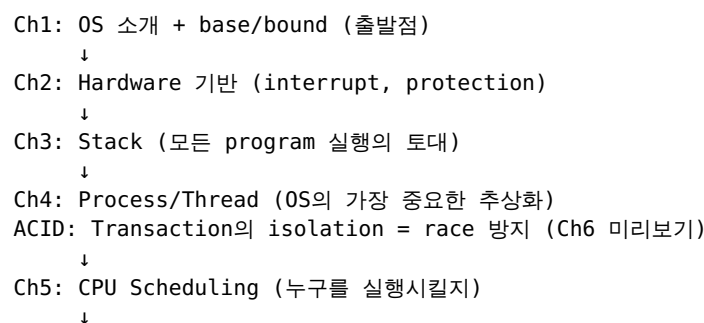
1. 3가지 **device type**: character, block, network
2. **Disk seek/rotation**이 **transfer**의 **1000배** 느림
3. **Flash**는 **erase-before-write + wear-out** → log-structured FS
4. **Device file**: major (driver) + minor (instance)
5. **Block driver**는 **buffer cache** — character와의 핵심 차이
6. **Two-level interrupt**: top half + bottom half
7. **I/O 4 조합**: polling/interrupt × blocking/non-blocking

⚠ 흔한 오해

- “**Disk**는 무조건 **SSD**보다 느림”: Sequential read는 비슷. random에서 차이.
- “**Buffer cache**는 항상 좋음”: 큰 sequential write는 DirectIO가 나옴.
- “**Polling**은 항상 나쁨”: 매우 빠른 device에서는 효율적.

📖 전체 요약

12개 챕터의 큰 그림



Ch6: Synchronization (race 해결)
↓
Ch7: Deadlocks (synchronization의 부작용)
↓
Ch8: Linker (.o → executable)
↓
Ch9: Dynamic Memory (heap, malloc)
↓
Ch10: Segmentation/Paging (modern MMU)
↓
Ch11: Demand Paging (virtual memory의 환상)
↓
Ch12: Device Drivers (I/O abstraction)

핵심 진화 트리들

Memory protection: Base/Bound → Multiple Segments → Paging → Paged Segmentation + TLB → Multi-level (x86-64 4-level)

Scheduling: FIFO → SJF → RR → MLFQ → GPS → WRR/WFQ → CFS

Synchronization 추상화: Disable interrupt → atomic instruction (TAS) → Semaphore → Condition Variable → Monitor

Memory management: Stack (LIFO) → Heap (any order) → Demand paging (lazy) → mmap (file integration)

모든 챕터를 관통하는 원칙들

1. 추상화는 비용을 동반 — 항상 trade-off 존재
2. **Hardware**와 **OS**의 협력 — 모든 OS 기능은 hardware 지원에 의존
3. **Locality**가 성능의 비밀 — caching, paging, scheduling 모두 활용
4. **Lazy is good** — fork COW, demand paging, lazy linking
5. **Mechanism**과 **policy** 분리 — dispatcher vs scheduler, file system vs disk scheduler

끝.